

LAB 5 Report

- 1. In the notebook, the audio waveform is transformed into time-frequency features before being passed to the keyword spotting model. What is the purpose of converting raw audio into spectrogram/MFCC-style features instead of directly using the raw time-domain waveform?**

In order to look at the image representation of the waveform, raw audio is converted to a spectrogram/MFCC. It is easier for the model to observe and learn patterns from the time-frequency pixel values compared to the raw time-domain waveform. Spectrograms capture the energy at different frequencies over time, which directly encodes a lot of information about different keywords, something the raw waveform makes very difficult to extract. Since the audio varies in length, converting to a fixed-size 2D feature map ensures the model always receives a consistent input shape. This also significantly compresses the data, for example, a 1-second audio clip with thousands of raw samples is reduced to just around 13 MFCC values, making it practical for a microcontroller. MFCC features are also more robust to background noise and amplitude variation than raw waveforms which improves accuracy.

- 2. During full integer quantization, the notebook uses a representative dataset. What is the purpose of the representative dataset, and why is it important for creating an INT8 TensorFlow Lite model?**

During full integer quantization, the TensorFlow Lite converter has to map all floating-point tensors like inputs, activations, and outputs to 8-bit integers. To do this, it needs the minimum and maximum range of values at every tensor, which it cannot determine from weights alone.

The representative dataset provides some randomly chosen samples from the training/evaluation set that are run through the model during conversion to compute the scale factor and zero-point for every tensor. Here, we receive MFCC/mel spectrogram features rather than raw audio, the representative dataset must supply those same preprocessed features.

Without it, scale calibration fails, causing value clipping off and low accuracy. With it, the INT8 model preserves float32 accuracy while being small enough to be deployed on microcontrollers.

3. What does `audio_processor.get_data()` do? Refer to `input_data.py`.

The `get_data()` method selects samples from the one of split testing, training, or validation folds, loads the corresponding .wav files, applies background noise mixing and time shifting augmentation according to the constants specified above, then runs the TensorFlow preprocessing graph via `sess` to convert raw waveforms into MFCC features. It returns data and/or labels where data is the MFCC feature array and labels are integer class indices. There are 2 different calls to it. The first gets 1 sample at a time for quantization calibration in the representative set, discards the label, and reshapes output for the TFLite converter. The second gets all testing samples at once for evaluation, keeps labels for accuracy calculation, and adds a dimension through `np.expand_dims` to match the architecture's expected input shape.

4. In the final model export step, the TensorFlow Lite model is converted into a C/C++ byte array. What are `g_model` and `g_model_len`, and why do they need to be copied correctly into the Arduino source file?

`g_model` is a `const unsigned char[]` array containing the raw bytes of the .tflite model file encoded as hex values. It includes the entire quantized INT8 model like its weights, graph structure, and quantization parameters which are to be stored in the Arduino's memory. `g_model_len` is the total number of bytes in `g_model`, used by the TFLite Micro interpreter at runtime to know how many bytes to read when loading the model.

They have to be copied correctly into the Arduino source file because the TFLite Micro runtime references these variables by name in the source code. If any values in `g_model` are wrong/missing, the interpreter loads a bad model and either crashes/produces wrong predictions. If model length is not correct the interpreter reads too few/many bytes, causing undefined behavior and error in placing it in arduino's memory.

5. Please include a screenshot showing the correct identification of at least one keyword, either "YES" or "NO", as displayed in the Arduino IDE Serial Monitor after deploying your KWS model on the Arduino Nano 33 BLE. The screenshot should be included in your submitted PDF.

YES

Heard unknown (212) @122736ms
Heard yes (207) @126608ms
Heard unknown (204) @133424ms
Heard unknown (210) @135632ms
Heard unknown (229) @137200ms
Heard unknown (201) @140048ms

NO

Heard unknown (209) @17632ms
Heard unknown (222) @19200ms
Heard no (208) @23072ms
Heard unknown (205) @23984ms
Heard unknown (205) @27120ms
Heard unknown (205) @28880ms